CÓDIGOCLARO · MATERIAL DE ESTUDIO

Simulacros DAM: integración de módulos

Casos de examen con XML, XPath, SQL, PL/SQL, Java y desarrollo web

1. Preparar un simulacro integrado

Objetivo: Organizar un ejercicio de varias asignaturas antes de comenzar a programar y entregar evidencias verificables.

Un caso integrado suele describir una entidad, datos de entrada, almacenamiento y una interfaz. Primero identifica requisitos y entrega cada resultado en el formato solicitado: esquema, consulta, rutina, programa o formulario. Dibuja las relaciones y define claves antes de escribir SQL. Anota los datos límite que podrían romper una solución: lista vacía, identificador inexistente, XML sin namespace esperado, número inválido o respuesta HTTP fallida. Divide el tiempo en lectura, implementación, ejecución y revisión. Al terminar, presenta evidencia reproducible: datos de prueba, entrada, salida esperada y explicación breve de la decisión más importante.

Código · ejemplo

```
[Plan de trabajo]
1. Extraer entidades y reglas
2. Diseñar claves y formatos
3. Implementar por módulos
4. Probar casos normales y frontera
5. Revisar la salida y documentar
```

Práctica: El caso da cursos, alumnado y matrículas. ¿Qué resultado planeas antes de abrir el editor?

Respuesta razonada: Lista entidades, relaciones y operaciones pedidas; anota una fila válida y un caso sin matrícula. Decide qué datos están en XML, SQL y objeto Java, y cómo se identifica cada entidad. Ese pequeño contrato orienta el modelo y previene que cada módulo use nombres o tipos distintos.

2. XML de matrícula y consulta XPath

Objetivo: Validar una entrada XML de matrículas y seleccionar los registros cuyo estado y nota cumplan el enunciado.

Supón que el documento usa el namespace `urn:dam:matriculas` y contiene alumnos con matrículas. El cuerpo de la ruta y los predicados usa XPath 3.1; como el ejemplo incluye `declare namespace`, el bloque completo es una consulta XQuery 3.1. Si se ejecuta solo la expresión con una API XPath, elimina el prólogo y enlaza `m` en el contexto estático; configura también `xs` si el procesador no lo ofrece. Antes de consultar, comprueba que el XML esté bien formado y que cada matrícula incluya identificador de módulo, estado y nota cuando el estado sea evaluado. XPath debe enlazar la URI a un prefijo propio del contexto estático. Una ruta absoluta concreta evita seleccionar nodos de otros vocabularios. Si se usa posición, define el orden; no confundas el primer resultado documental con el más reciente. En una respuesta de examen, muestra el resultado y explica cómo respondería la consulta si no hubiera coincidencias.

Código · ejemplo

```
[XQuery 3.1]
declare namespace m = "urn:dam:matriculas";
//m:matricula[@estado = "evaluada"][xs:decimal(m:nota) ge 5]
```

Práctica: ¿Qué ocurre si `nota` contiene `apta` en lugar de un decimal?

Respuesta razonada: La conversión directa a decimal falla. El contrato debería restringir el valor mediante XSD o la consulta debe comprobar `castable as xs:decimal` y definir qué hacer con el dato no convertible. También debe confirmarse que la regla aplique solo a matrículas evaluadas; un estado pendiente no tiene por qué tener nota.

3. Modelo relacional y consulta de aprobados

Objetivo: Traducir alumnado, módulos y matrículas a claves relacionales y consultar datos sin duplicados accidentales.

Un diseño normalizado separa alumno, módulo y matrícula; la matrícula actúa como entidad asociativa cuando un alumno cursa varios módulos y cada módulo tiene muchos alumnos. Claves primarias identifican filas y claves foráneas preservan la relación. Declara si existe una sola matrícula por pareja alumno-módulo con una restricción única. Una consulta con JOIN debe especificar las columnas de unión, no confiar en coincidencias de nombres. `DISTINCT` puede ocultar una unión incorrecta, por lo que primero confirma cardinalidades. Usa aliases y selecciona solo campos necesarios. El SQL de ejemplo emplea sintaxis estándar cercana a Oracle; adapta tipos a la versión del motor.

Código · ejemplo

```
[SQL · Oracle AI Database 26ai]
SELECT a.nombre, m.nombre AS modulo
FROM alumno a
JOIN matricula x ON x.alumno_id = a.id
JOIN modulo m ON m.id = x.modulo_id
WHERE x.nota >= 5;
```

Práctica: ¿Cómo detectarías si el resultado duplica un alumno por tener varias matrículas aprobadas?

Respuesta razonada: Define la granularidad del resultado: aquí una fila representa una matrícula aprobada, no necesariamente una persona única. Si piden alumnos, usa `EXISTS` o una agrupación adecuada; no añadas `DISTINCT` sin decidir si se debe mostrar cada módulo. Prueba un alumno con dos módulos aprobados para observar la cardinalidad.

4. SQL: cambios atómicos de matrícula

Objetivo: Proteger una operación que registra una matrícula y descuenta una plaza disponible dentro de una transacción.

La operación modifica al menos la matrícula y el número de plazas. Si una parte se confirma y la otra falla, el estado queda incoherente. Agrupa los cambios en una transacción y define qué tabla es fuente de verdad para disponibilidad. Un patrón condicional actualiza solo si quedan plazas; revisa el número de filas afectadas antes de insertar. La restricción única evita matrículas duplicadas aun bajo concurrencia. Parámetros enlazados previenen errores de comillas y reducen riesgos de inyección. SQL de Oracle distingue secuencias y sintaxis de otros motores, así que usa el dialecto indicado en el examen.

Código · ejemplo

```
[SQL · Oracle AI Database 26ai]
UPDATE modulo
SET plazas = plazas - 1
WHERE id = :modulo_id AND plazas > 0;
-- comprobar SQL%ROWCOUNT antes de insertar
```

Práctica: ¿Cuándo insertas la matrícula y cómo respondes si se actualizaron cero filas?

Respuesta razonada: Inserta solo si la actualización afectó exactamente una fila. Si afectó cero, no quedaban plazas o el módulo no existe; no insertes la matrícula y devuelve el resultado de negocio correspondiente. La transacción y restricciones únicas protegen consistencia adicional; el bloqueo y nivel de aislamiento deben probarse con concurrencia real.

5. PL/SQL: cursor para emitir certificados

Objetivo: Recorrer matrículas aprobadas y producir una salida sin perder errores ni dejar la transacción implícita.

Una rutina que genera certificados puede consultar aprobados por curso y recorrer filas. Usa un cursor FOR LOOP si el recorrido es necesario y procesa una fila cada vez; no incluyas `COMMIT` por iteración. Si la salida solo es un conjunto de datos, una consulta SQL puede ser mejor que PL/SQL. Define qué ocurre con alumnos sin correo o certificados ya emitidos y protege la operación con una clave única o idempotencia. El cliente de examen debe indicar si espera `DBMS_OUTPUT`, filas devueltas o una tabla de emisión. Comprueba que el entorno habilite la salida antes de depender de ella.

Código · ejemplo

```
[PL/SQL · Oracle AI Database 26ai]
BEGIN
  FOR r IN (SELECT alumno_id FROM matricula WHERE nota >= 5) LOOP
    DBMS_OUTPUT.PUT_LINE('Aprobado: ' || r.alumno_id);
  END LOOP;
END;
/
```

Práctica: ¿Por qué este bloque no demuestra todavía que un certificado se haya guardado?

Respuesta razonada: `DBMS_OUTPUT` solo escribe una línea en el buffer de salida de la sesión; no inserta ni confirma un certificado. Para persistir, hace falta una tabla y una operación DML, con claves idempotentes y una política de transacción. La salida de consola sirve para revisión, no como prueba de persistencia.

6. Java: agrupar notas con colecciones

Objetivo: Calcular una media por alumno y tratar explícitamente faltantes, precisión y claves de colección.

Los mapas facilitan asociar a cada alumno una colección de notas; una solución clara puede usar `Map<Long, List<BigDecimal>>`. Define si las notas nulas se descartan o se rechazan y si deben estar en un rango. Con dinero o notas decimales, `BigDecimal` evita aproximaciones binarias, pero la escala y modo de redondeo siguen siendo decisiones del dominio. No asumas que el orden de un `HashMap` es estable. Un DTO inmutable ayuda a entregar resultado coherente. La prueba debe cubrir alumno sin notas, una nota y varias notas con media repetible.

Código · ejemplo

```
[Java 21]
BigDecimal average(List<BigDecimal> values) {
    return values.stream().reduce(BigDecimal.ZERO, BigDecimal::add)
        .divide(BigDecimal.valueOf(values.size()), 2, RoundingMode.HALF_UP);
}
```

Práctica: ¿Qué caso falla en este método si `values` está vacío?

Respuesta razonada: La división usa divisor cero y lanza `ArithmeticException`. Decide si no hay media debe producir `Optional.empty()`, un error de dominio o una convención documentada; no inventes cero, pues cero sería una media real. Valida también nulos y fija escala y redondeo.

7. HTML: formulario accesible de matrícula

Objetivo: Crear un formulario semántico que asocie etiquetas, valide datos básicos y comunique errores con claridad.

Un formulario requiere etiquetas vinculadas a controles, nombres descriptivos y botones cuyo propósito esté claro. Los atributos HTML como `required`, `type="email"` o `min` ayudan al usuario, pero no sustituyen validación en servidor. Para accesibilidad, asocia `label` por `for`/`id`, comunica instrucciones y errores sin depender solo del color, y conserva foco utilizable con teclado. El botón de envío debe tener tipo explícito cuando hay otros botones en el formulario. No pidas datos que la operación no necesite. Prueba con lector de pantalla o navegación por teclado y con datos inválidos.

Código · ejemplo

```
[HTML]
<label for="email">Correo del alumno</label>
<input id="email" name="email" type="email" required>
<button type="submit">Matricularme</button>
```

Práctica: ¿Por qué el servidor debe volver a validar un correo marcado como obligatorio en HTML?

Respuesta razonada: La persona puede desactivar o saltarse las restricciones del navegador y enviar una petición modificada. El servidor es la autoridad que valida formato, tamaño, autorización y reglas de negocio. La validación del cliente mejora la experiencia y da errores tempranos; no es una frontera de seguridad.

8. JavaScript: petición y estados de interfaz

Objetivo: Solicitar datos de módulos con `fetch`, verificar estado HTTP y representar carga, error y éxito.

Una interfaz que consulta plazas debe distinguir tres estados visibles: cargando, resultado y error. `fetch` devuelve una promesa; debe esperarse la respuesta y comprobar `response.ok`, porque 404 o 500 no siempre rechazan esa promesa. Después se analiza JSON y se valida su forma antes de mostrar campos. Un `try/catch` contextualiza fallos de red, estado HTTP y análisis. Evita insertar texto remoto con `innerHTML`; usa `textContent` si solo quieres mostrar texto. Si pueden solaparse solicitudes, considera cancelación con `AbortController` para evitar que una respuesta antigua reemplace una nueva.

Código · ejemplo

```
[JavaScript · ECMAScript 2025]
async function cargarModulos() {
  list.textContent = 'Cargando…';
  try {
    const response = await fetch('/api/modulos');
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const modulos = await response.json();
    if (!Array.isArray(modulos) ||
        !modulos.every(m => m && typeof m.nombre === 'string')) {
      throw new Error('Formato inválido');
    }
    list.textContent = modulos.length
      ? modulos.map(m => m.nombre).join(', ')
      : 'No hay módulos.';
  } catch {
    list.textContent = 'No se pudieron cargar los módulos.';
  }
}
void cargarModulos();
```

Práctica: ¿Qué riesgo hay en asumir que `modulos` es siempre un array válido?

Respuesta razonada: La respuesta puede tener otro formato, estar vacía o provenir de un servidor con error de contrato. Comprueba `Array.isArray` y valida los campos requeridos antes de usarlos. Mantén salida segura con `textContent`, muestra un mensaje útil en el estado de error y no expongas detalles internos del servidor.

9. Depurar una integración entre módulos

Objetivo: Seguir un dato desde la entrada hasta el resultado y localizar el primer punto donde cambia incorrectamente.

Cuando un certificado muestra una nota equivocada, traza el mismo identificador en XML, SQL, PL/SQL, Java y la interfaz. Comprueba en cada frontera tanto representación como tipo: `8.5` como decimal, cadena JSON o `BigDecimal` no son intercambiables sin conversión. Añade un caso mínimo de reproducción y registra solo datos permitidos. Verifica que el JOIN no multiplique filas, que el cursor reciba las condiciones correctas y que la API devuelva el HTTP esperado. Aísla fallos por capas antes de modificar varias piezas a la vez. Después de corregir, añade una prueba de regresión que falle con el defecto anterior.

Código · ejemplo

```
[Lista de comprobación]
Entrada → validación → consulta → resultado → serialización → presentación
```

Práctica: La base de datos tiene la nota correcta, pero la web muestra otra. ¿Qué fronteras revisarías primero?

Respuesta razonada: Compara el registro por su ID en la consulta y su proyección, revisa la conversión a DTO y el JSON serializado, y observa qué respuesta recibió el navegador. Si el servidor devuelve el dato correcto, inspecciona estado y renderizado del cliente; si no, retrocede al JOIN y a la condición SQL. Cambia una sola frontera y reproduce.

10. Entrega del simulacro y autoevaluación

Objetivo: Presentar código reproducible y revisar corrección, robustez, claridad y adecuación al dialecto solicitado.

La entrega debe incluir los archivos pedidos y una explicación breve de decisiones. Anota motor y versión para SQL y PL/SQL, versión Java y navegador o runtime si afectan compatibilidad. Ejecuta pruebas, incluye datos mínimos reproducibles y muestra salidas esperadas. Revisa que no haya credenciales, rutas personales ni información de alumnado en el repositorio. La autoevaluación puede puntuar cumplimiento del contrato, casos límite, claridad del modelo, manejo de errores y evidencia de ejecución. Un punto parcial razonado vale más que ocultar una decisión no resuelta; identifica su supuesto y explica cómo lo verificarías con la documentación oficial.

Código · ejemplo

```
[Rúbrica]
Contrato 30 % · datos y consultas 25 % · implementación 25 % · pruebas y
explicación 20 %
```

Práctica: ¿Qué evidencia distingue una consulta correcta de una consulta que solo compila?

Respuesta razonada: Incluye datos de prueba con cardinalidades conocidas, resultado esperado y resultado observado. La compilación detecta sintaxis y algunos tipos, no valida el significado del JOIN ni los filtros. Contrasta un caso normal, uno sin coincidencias y uno con duplicidad o límite; explica la diferencia respecto al contrato.

Fuentes oficiales

W3C · XML 1.0 · W3C · XPath 3.1 · Oracle AI Database 26ai · SQL Language Reference · Oracle AI Database 26ai · PL/SQL Language Reference · Oracle · Java Language Specification 21 · WHATWG · HTML Living Standard · ECMA · ECMAScript 2025 · W3C WAI · Forms · W3C · XQuery 3.1